Beyond Context Compression: State Composition and Recent History in Coding Agents

Alex Markov-Buturskiy

Abstract

Structured state offers a way to bound a coding agent's active context without carrying its full interaction history. Whether this also reduces the cost of solving tasks depends on what the replacement memory preserves. We evaluate SKILL.state-inspired context policies on Terminal-Bench, varying state ownership, retained evidence, recent history, update protocol, and schema. Model-written state reduces the median peak-prompt ratio to 0.47 on shared successes, yet raises the cost ratio to 2.57 and lowers overall task success from 61.3% to 34.9%. An exploratory comparison of hybrid memory, which separates runtime records from model-written interpretation, associates richer execution records with better success and lower overhead than file references alone. Further enrichment does not remove the full-history deficit. Retaining recent interactions gives the clearest recovery: model-written and hybrid policies reach 59.0% and 48.3%, against their respective controls of 61.3% and 51.7%. The economic outcomes differ: cost per observed success falls below the control for model-written memory but remains above it for the hybrid variant. The tested schema and protocol changes do not produce comparable recovery. These results support evaluating structured memory through the evidence it retains and the work needed to finish a task, rather than compression alone. Recent-history retention is a measured partial remedy; the incremental benefit of state over the same history window remains open.

1 Introduction

A coding agent accumulates evidence as it works. A command exposes a dependency, an edit changes the meaning of an earlier observation, and a test rules out a proposed fix. An interaction transcript preserves these events together with their context. Replacing that transcript with compact state requires deciding which information later decisions will need. The consequences extend beyond the length of the next prompt.

SKILL.state makes this replacement an explicit runtime design [Badhe et al., 2026]. Persistent instructions, mutable structured state, and the current observation replace an expanding conversation. The attraction is clear: when those inputs are bounded, context need not grow with the trajectory. But a bound on context is not a bound on the work required to solve the task. A smaller representation may omit evidence, require additional state-writing output, or change how the agent acts and stops. Cache-aware pricing further separates prompt size from monetary cost.

We study this tension in a terminal-based coding agent. The evaluation transfers the state-based runtime idea to a different model, agent, and benchmark; it is not a reproduction of the original study. Our central question is what information must survive history replacement for compression to remain useful over a complete task.

The comparisons separate decisions that are easily conflated under agent memory. First, who writes the memory? A model can maintain a semantic account of its work, while the runtime can retain observed tool events directly. This motivates hybrid state: execution records and model-written interpretation have different owners. Second, what does the runtime retain?

File references, modifications, command outcomes, and diagnostics convey different amounts of execution evidence. Third, which original interactions remain available? A recent-history window preserves detail without requiring the state writer to anticipate its relevance. Finally, the update protocol and schema determine how the model expresses its semantic memory.

The results connect these design choices to task success and resource use. Model-written state substantially reduces peak prompts while lowering success and increasing conditional successful-execution cost. An exploratory component comparison shows why hybrid memory cannot be treated as a single intervention: richer execution records perform better than file references alone, but additional records do not yield continuous improvement. Retaining recent interactions produces the clearest recovery in both state families. The tested schema and protocol changes do not.

These findings motivate evaluating structured state together with its retained evidence and surrounding history. We distinguish partial recovery from a complete replacement for history, and separate measured results from proposed adaptive switching or archival retrieval. The practical outcome is a set of quality–cost trade-offs, not a universal recommendation to discard or retain every transcript.

2 Context policies and research questions

History and state differ in what they preserve. History retains records in their original order. State rewrites selected details into fields that describe the current task. A state field may record that a test failed without preserving its complete output or the edits preceding it. Conversely, keeping those edits does not guarantee that a model will use them effectively.

There are also different ways to produce state. A model can maintain semantic content such as hypotheses and pending work. A runtime can record facts directly observable as the agent works, such as completed commands and their outcomes. We call a state that combines these sources *hybrid state*. Runtime-derived records reduce the need to restate observable events, but do not by themselves explain what those events mean or what the agent should do next.

The published SKILL.state contract jointly emits a state patch and an action [Badhe et al., 2026]. A separate state-update tool changes that interface: the agent updates memory through a tool call within its main loop. Our inline policy emits the patch and action together, while the schema comparison tests how the patch is organized. These changes test specific aspects of the port, not every way in which it differs from the published system.

We first ask whether prompt compression remains beneficial after accounting for task success, cumulative generation, and cost. We then examine what the runtime should record, whether recent interactions recover capability beyond those records, and whether schema or protocol changes offer an alternative remedy. Each comparison uses a within-family control. Adding recent history to state tests the value of that history within a state-based agent; it does not isolate the value of the state block itself.

3 Context Policies and Update Protocols

Let P denote task instructions, o_t the current observation, and $H_{<t}$ preceding observable messages and tool results. Full history uses

$$C_t^{\text{full}} = [P, H_{<t}, o_t]. \tag{1}$$

Structured memory instead constructs

$$C_t^{\text{state},k} = [P, \Sigma_t, \tau_k(H_{<t}), o_t], \qquad \tau_0(H) = \varnothing, \tag{2}$$

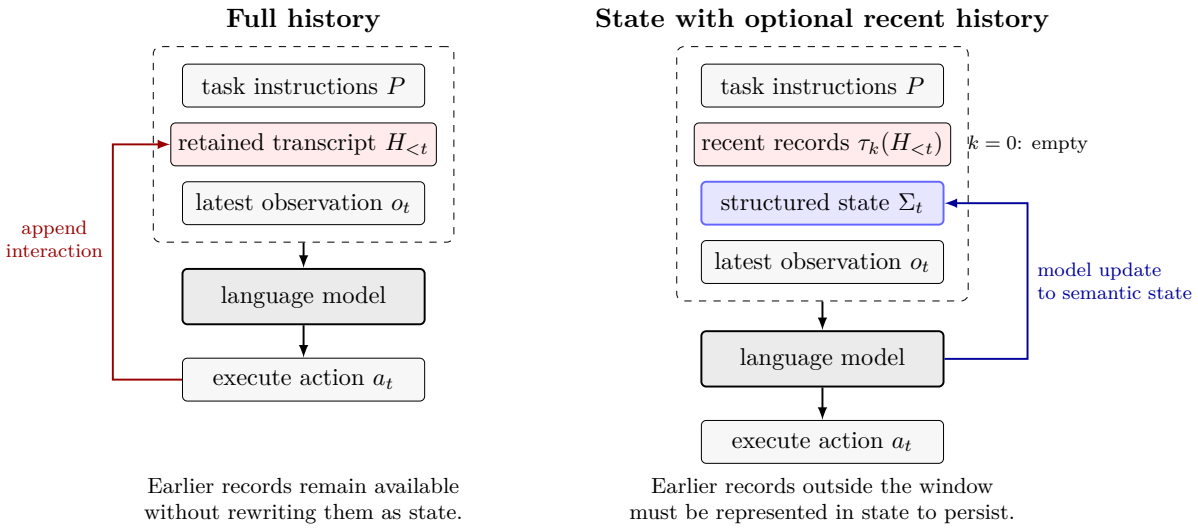


Figure 1: Context policies. State replaces the older transcript, while the retention parameter k controls the recent suffix. The arrows show information flow, not the number of model calls.

where τ_k selects available recent records and $\tau_\infty(H) = H$. Thus $k = \infty$ retains state *and* the transcript; the no-state full-history control is a different policy (Figure 1). Neither definition assumes access to hidden reasoning.

3.1 State ownership and update protocol

Model-written memory maintains semantic state S_t. Hybrid memory adds runtime-owned records V_t:

$$\Sigma_t = (S_t, V_t), \qquad S_{t+1} = \text{Apply}(S_t, \Delta S_t), \qquad V_{t+1} = \text{Record}(V_t, a_t, r_t). \tag{3}$$

The model maintains hypotheses and pending work; the runtime records observed tool events r_t, from file references to modifications, command outcomes, diagnostics, tests, and failed attempts. A semantic patch cannot edit V; this does not guarantee complete capture. Recent history can accompany either family.

A separate update tool accepts state patches; the inline protocol emits $\{\Delta S_t, a_t\}$ together, following the published SKILL.state contract [Badhe et al., 2026]. Algorithms 1 and 2 are reference contracts, not reconstructed traces.

Algorithm 1 Context selection and state ownership

Require: Instructions P, policy π, retention setting k, update protocol q, initial observation o

Initialize history H, semantic state S, runtime record V
while the task has not terminated **do**
  if $\pi =$ FullHistory **then**
    $C \leftarrow [P, H, o]$
  else
    $\Sigma \leftarrow S$ **or** (S, V) according to state ownership
    $C \leftarrow [P, \Sigma, \tau_k(H), o]$
  end if
  $(S', a) \leftarrow \text{PolicyStep}(C, S, \pi, q)$
  $r \leftarrow \text{Execute}(a)$; $S \leftarrow S'$
  if hybrid memory is enabled **then**
    $V \leftarrow \text{Record}(V, a, r)$
  end if
  Append a record of (o, a) to H; $o \leftarrow r$
end while

Algorithm 2 Inline patch–action contract

Require: Current context C and semantic state S

$(\Delta S, a) \leftarrow \text{Model}(C)$
if $\text{Valid}(\Delta S, S)$ **then**
  return $(\text{Apply}(S, \Delta S), a)$
else
  Keep S unchanged and return a validation observation
  Do not execute a from the rejected envelope
end if

3.2 Context bounds and task cost

Full history yields $O(T^2)$ cumulative input over T actions; bounded state and a fixed window yield $O(T)$. Neither bound fixes T itself. With cumulative uncached input U, cache reads K, output O, and tariffs α, γ, β,

$$D = \alpha U + \gamma K + \beta O. \tag{4}$$

State maintenance can add output, reduce cache reuse, or change T, so a smaller peak prompt does not determine D.

4 Experimental methodology

4.1 Agent and comparisons

We evaluate `zai/glm-5.3-flash` with Pi 0.85.1 and Harbor 0.22 on Terminal-Bench 2.0 [Merrill et al., 2026], with a 200-turn limit. Success is the binary task-verifier outcome. The analyzed subset contains 87 tasks; two tasks whose environments could not initialize are excluded.

Four comparisons are reported, each with its own full-history control matched on task and repetition; controls are never pooled.

- *Model-written state* uses four conditions and three repetitions, $k \in \{0, 2, 6\}$.
- *Hybrid state* uses eight conditions and two repetitions. Variants remove failure records, add recent history, or cross tool/inline updates with existing/task-adapted schemas.

- *Runtime-record contents* uses four bundled configurations and a control with one repetition. Its control and richest rows overlap the first hybrid repetition, so it is not an independent replication.
- *Window pilot* uses 12 tasks, six conditions ($k \in \{0, 2, 6, 12, \infty\}$ and control), three repetitions. Only finalized summaries survive.

Schema adaptation used the same tasks, and weaker component configurations were dropped after preliminary runs, so these are exploratory, in-sample comparisons. Across implementations k is the configured retention setting; equal k does not guarantee identical serialization.

4.2 Metric contract

Every comparison uses the same two panels with identical rows. The *quality panel* reports solved rate over all retained outcomes, median turns T, median normalized cost, median matched cost ratio to full history, and successes per dollar $\sum Y / \sum D$ (failures included). The *resource panel* reports median peak prompt, cumulative uncached input U, cache reads K, output O, and cache hit

$$h_{iar} = \frac{K_{iar}}{U_{iar} + K_{iar}}, \tag{5}$$

summarized as a per-execution median rather than a ratio of marginal token medians. No cache-write counter exists. Normalized cost is

$$D = 10^{-6}(0.15U + 0.03K + 0.50O) \tag{6}$$

in US dollars, not a provider invoice. Metrics that the surviving records cannot reconstruct (hybrid peaks and cache hit, pilot solved rates and input counters) are shown as –, never imputed.

4.3 Statistical analysis and cohorts

Matched outcomes use two-sided exact McNemar tests; model-written contrasts add 95% task-bundle bootstrap intervals (100,000 resamples, seed 20260922) that resample tasks with all conditions and repetitions. Tests are unadjusted and exploratory; non-significance is not equivalence. Horizon cohorts use the control's median turns, giving short $T \leq 8$ (31 tasks), medium $8 < T \leq 20$ (29), long $T > 20$ (27), fixed across conditions. They describe control behavior, not assigned difficulty. The pilot repetition diagnostic is tool calls divided by distinct tool/argument combinations; 1.00 means no identical re-invocation, not absence of semantic redundancy. Supplementary S1 lists the available material.

5 Compression is not complete-task efficiency

Model-written state achieves compression but does not preserve task performance. On shared successes its median peak-prompt ratio is 0.47, while its cost ratio is 2.57 and its output-token ratio is 4.55. Across all retained outcomes, success falls from 61.3% to 34.9%, with a task-bootstrap difference interval of $[-34.1, -18.8]$ percentage points (Supplementary S2).

Cache hit exposes a second cost of replacing history. The median fraction of input served from cache is 83.3% for full history, 49.4% for state alone, 37.4% at $k = 2$, and 20.3% at $k = 6$. A growing transcript can reuse an unchanged prefix, whereas changing state and a moving window can reduce that reuse. The measurements support different cache behavior, not attribution to a particular serialization detail.

A low cache-hit fraction is not itself a verdict against a memory policy. The recent-history window with the lowest hit rate also gives the best success among the tested state variants. Task-level cost combines cache reuse with uncached input, generated output, and the number

of actions. For state alone, cost per observed success is $0.1630, against $0.1116 for full history. The following comparisons retain the same quality, money, token, and cache columns so that changes in representation can be assessed on all these dimensions.

6 What should the runtime remember?

Hybrid state lets the runtime keep observed command outcomes while the model maintains hypotheses and pending work (Figure 2). Its value depends on what the runtime records: which file was seen differs from what changed and which commands followed.

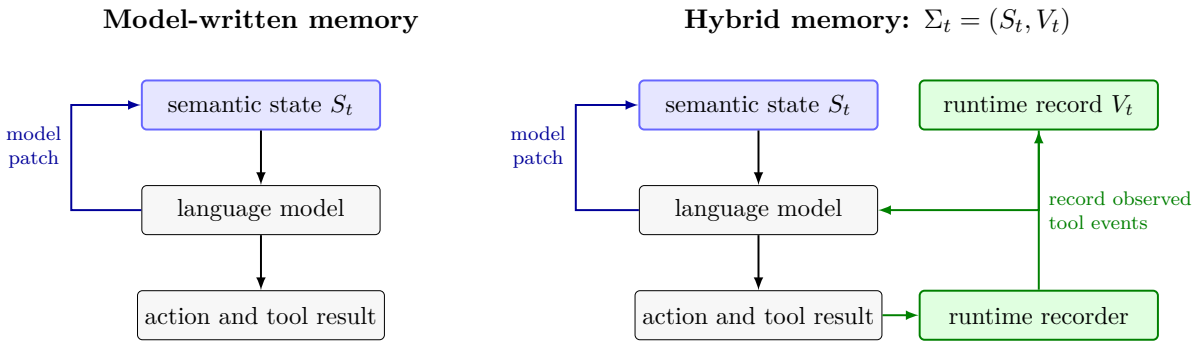
6.1 From file references to execution evidence

File references alone reach 28.7% solved (Table 1). Adding modifications and command records raises this to 40.2%, lowers median output from 23.2 to 16.5 thousand tokens, and lowers median cost from $0.01406 to $0.01016. Diagnostics, environment, and tests give 42.5%; failed-attempt records give 41.4%. None reaches full history at 55.2%. The gain sits between file references and execution records; further enrichment plateaus (Figure 3). Rows change bundles of records, so no single component is isolated.

Every runtime-record configuration has a lower cache hit than full history (42.1–54.6% against 82.7%), consistent with a changing state block breaking prefix reuse that a growing transcript keeps. Successes per dollar rise from 12.87 to 20.79, approaching but not exceeding the control's 22.39. On tasks solved by every configuration, the richest variants cost 1.03 and 1.17 times full history (Supplementary S4): richer records beat a sparse index, but show no cost advantage over the transcript.

6.2 Do failure records close the remaining gap?

In the final hybrid comparison, failure records move success from 38.5% to 39.7%, against 51.7% for full history (Table 2); the matched difference is small. A stored outcome need not preserve the order and context a later decision needs. More fields did not recover the gap; retaining some original interaction is the remaining candidate, tested next.



The model updates S; the runtime updates V.
A runtime record preserves captured evidence, not a complete account of the environment.

Figure 2: State ownership. Hybrid memory retains model-written semantic content and adds runtime-derived records. Recording an event does not establish that an extracted interpretation is correct or that all relevant events were captured.

Table 1: Runtime-record contents under the same semantic schema (exploratory, one repetition). Top: quality panel over all retained outcomes. Bottom: resource panel, per-execution medians. Rows beginning with + add to the preceding row.

Approach	Solved rate (%)	Median T	Median cost ($)	Cost ratio (×)	Solved/$
Full history	55.2	11	0.01139	1.00	22.39
File references	28.7	15	0.01406	1.03	12.87
Files, changes, commands	40.2	9	0.01016	0.98	20.79
+ diagnostics, environment, tests	42.5	9	0.01316	1.01	20.92
+ failed-attempt records	41.4	9	0.01196	1.01	21.19

Approach	Peak prompt (ktok)	Uncached in (ktok)	Cached in (ktok)	Output (ktok)	Cache hit (%)
Full history	13.4	20.4	61.2	16.4	82.7
File references	3.9	15.9	17.9	23.2	54.6
Files, changes, commands	5.3	12.8	14.7	16.5	42.1
+ diagnostics, environment, tests	5.8	16.8	16.4	19.1	42.4
+ failed-attempt records	5.6	13.3	15.6	18.6	43.2

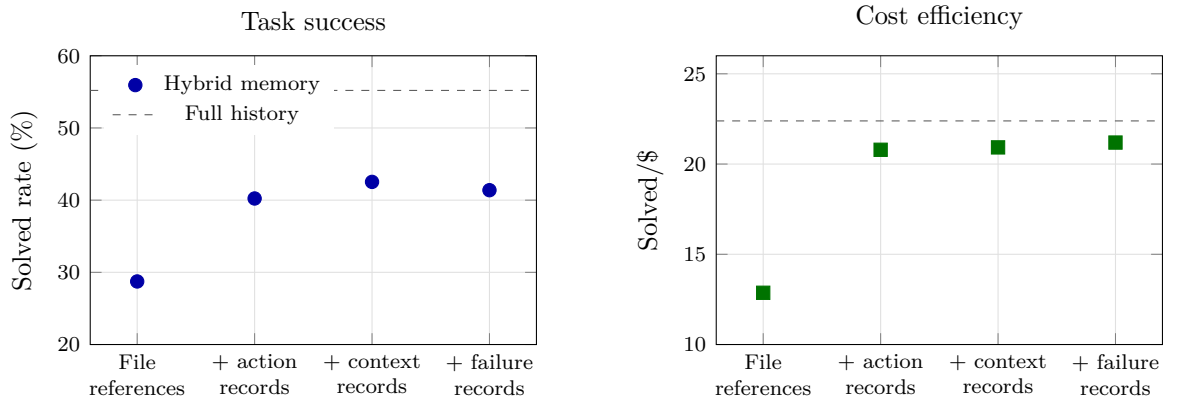


Figure 3: Exploratory comparison of cumulative runtime-owned information. Each position adds records to the preceding configuration: action records cover file changes and commands; context records cover diagnostics, environment, and tests. Efficiency includes the cost of unsuccessful episodes. Dashed lines show the same full-history reference. These descriptive observations share their richer configurations and reference with the first repeat of the final comparison; they do not isolate individual components or establish a monotonic benefit.

Table 2: Final hybrid comparison with and without failure records. Same two panels as Table 1. Hybrid aggregates cannot reconstruct matched cost ratios, peak prompt, or cache hit (–).

Approach	Solved rate (%)	Median T	Median cost ($)	Cost ratio (×)	Solved/$
Full history	51.7	11	0.0104	–	21.13
Hybrid, no failure records	38.5	8	0.0127	–	20.12
Hybrid state	39.7	8	0.0120	–	21.36

Approach	Peak prompt (ktok)	Uncached in (ktok)	Cached in (ktok)	Output (ktok)	Cache hit (%)
Full history	–	20.3	62.5	16.4	–
Hybrid, no failure records	–	16.6	10.2	19.1	–
Hybrid state	–	14.5	9.3	17.3	–

7 How much recent history is enough?

7.1 Capability recovery and its price

State records selected facts; recent history also retains the wording, ordering, and surrounding context of interactions. Adding it raises model-written success from 34.9% at $k = 0$ to 45.2% at $k = 2$ and 59.0% at $k = 6$. The last setting differs from full history by -2.3 percentage points, with interval $[-7.7, +3.4]$. This is substantial recovery, not demonstrated equivalence.

Table 3: Recent-history retention, quality and economics. All retained outcomes contribute. Each memory family has its own full-history control. Cost ratio is the median matched ratio, not a ratio of medians; unavailable hybrid ratios are left blank. Solved/$ uses aggregate cost, including failures.

Approach	Solved rate (%)	Median T	Median cost ($)	Cost ratio (×)	Solved/$
Model-written memory					
Full history	61.3	13	0.0137	1.00	8.96
Model-written state	34.9	25	0.0239	1.59	6.14
State + history ($k = 2$)	45.2	14	0.0151	1.14	9.16
State + history ($k = 6$)	59.0	11	0.0154	1.05	13.02
Hybrid memory					
Full history	51.7	11	0.0104	–	21.13
Hybrid state	39.7	8	0.0120	–	21.36
Hybrid + history ($k = 2$)	40.8	8	0.0122	–	21.65
Hybrid + history ($k = 6$)	48.3	9	0.0144	–	19.95

Table 4: Resource panel for the same history policies. Peak prompt and token counts are execution-level medians; cache hit is the median within-execution cached fraction. Hybrid peak and cache-hit statistics are unavailable, not zero.

Approach	Peak prompt (ktok)	Uncached in (ktok)	Cached in (ktok)	Output (ktok)	Cache hit (%)
Model-written memory					
Full history	16.1	28.8	87.4	16.4	83.3
Model-written state	4.0	24.9	36.7	37.4	49.4
State + history ($k = 2$)	6.2	30.3	19.1	20.1	37.4
State + history ($k = 6$)	11.3	53.3	14.8	16.6	20.3
Hybrid memory					
Full history	–	20.3	62.5	16.4	–
Hybrid state	–	14.5	9.3	17.3	–
Hybrid + history ($k = 2$)	–	22.4	6.4	16.6	–
Hybrid + history ($k = 6$)	–	49.0	9.9	16.4	–

The hybrid policy shows the same direction: 39.7% without history, 40.8% at $k = 2$, and 48.3% at $k = 6$. The latter improves on hybrid state alone by +8.6 percentage points ($p = 0.011$), against a full-history reference of 51.7%. Its cost per observed success increases from \$0.0468 to \$0.0501. For model-written memory it instead falls from \$0.1630 to \$0.0768. The same retention decision therefore has different economic consequences.

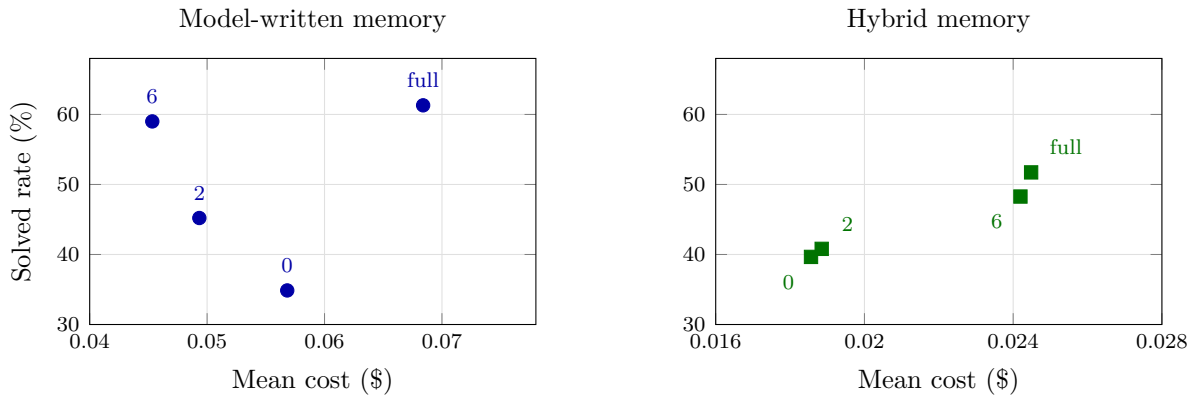


Figure 4: Success–cost trade-offs when retaining history. Labels are the retention setting k; full denotes the full-history reference without a state block. Each panel includes failures in mean cost and uses its own control. Hybrid mean costs use rounded reported totals. The horizontal scales differ.

7.2 The complete window sweep and why six

Recovery at $k = 6$ alone cannot show that a larger window adds nothing. The pilot therefore also ran $k = 12$ and $k = \infty$ (state plus the full transcript, distinct from the no-state control), with three repetitions on 12 tasks (Tables 5 and 6).

Table 5: Window pilot in the standard two panels (finalized medians over three repetitions). Per-execution solved rates, efficiencies, pooled cost ratios, and input counters were not retained (–). Cache hit is the reported pilot value, not reconstructed from counters.

Approach	Solved rate (%)	Median T	Median cost ($)	Cost ratio (×)	Solved/$
Full history	–	12	0.0035	–	–
Model-written state	–	25	0.0099	–	–
State + history ($k = 2$)	–	15	0.0054	–	–
State + history ($k = 6$)	–	12	0.0070	–	–
State + history ($k = 12$)	–	9	0.0059	–	–
State + history ($k = \infty$)	–	10	0.0060	–	–

Approach	Peak prompt (ktok)	Uncached in (ktok)	Cached in (ktok)	Output (ktok)	Cache hit (%)
Full history	9.87	–	–	6.54	86.5
Model-written state	3.46	–	–	29.62	64.3
State + history ($k = 2$)	6.18	–	–	14.20	48.8
State + history ($k = 6$)	8.43	–	–	12.16	27.1
State + history ($k = 12$)	11.34	–	–	7.93	27.0
State + history ($k = \infty$)	14.02	–	–	9.11	38.2

Three measurements stop changing at $k = 6$. Tasks solved rise from 9 of 12 at $k \leq 2$ to 12 at $k = 6$ and stay there; the repetition ratio falls from 1.27 to 1.00 and stays there; and the $k = 6$, 12, and ∞ rows each differ from the control by a single discordant task. Two of these flat lines are bounds, not plateaus. Solving all 12 tasks is the ceiling of the task set, and a ratio of one is the floor of the repetition diagnostic. A 12-task pilot needs at least six one-sided discordant tasks for significance at the five percent level, so it cannot separate conditions that are all at or near the ceiling. The honest reading is that the threshold lies between two and six messages, not that wider windows are useless.

Table 6: Window pilot outcomes reported in the research log (raw sessions unavailable). A task counts as solved if any of its three repetitions passed. Discordant tasks and exact McNemar p compare with full history. Output per turn is the reported median.

Approach	Tasks solved, % (any repeat)	Discordant (gain/loss)	McNemar p	Median calls / distinct calls	Output tok per turn
Full history	91.7	–	–	1.00	528
Model-written state	75.0	0/2	0.500	1.27	1105
State + history ($k = 2$)	75.0	0/2	0.500	1.06	1116
State + history ($k = 6$)	100.0	1/0	1.000	1.00	1053
State + history ($k = 12$)	100.0	1/0	1.000	1.00	1059
State + history ($k = \infty$)	100.0	1/0	1.000	1.00	967

The resource columns keep moving. Median peak prompt grows from 8.43 thousand tokens at $k = 6$ to 11.34 and 14.02; median turns fall from 12 to 9 and 10; median cost is \$0.0070, \$0.0059, and \$0.0060. Matched cost ratios stay between 1.55 and 2.25 for every $k \geq 6$ in every repetition (Supplementary S5). The reason is visible in output per turn. Every state condition writes about twice the control's 528 tokens (1053 at $k = 6$), because the model rewrites state on each step. Widening the window cannot remove that fixed overhead; it only trades fewer turns for larger prompts. Cache hit also stays low (27.1–38.2% against 86.5%), consistent with a state block that precedes the transcript in Eq. (2) and changes every turn.

So $k = 6$ is the smallest tested window at which success and redundant re-invocation reach the pilot's resolution limit, and the cheapest point beyond it is not identified by these data. The main model-state comparison confirms recovery at $k = 6$ but did not run $k = 12$ or $k = \infty$; a discriminating follow-up needs harder tasks with headroom above the ceiling and a no-state recent-history control.

8 Where trajectory length changes the trade-off

The aggregate result hides substantial differences across task trajectories. We use the control-defined cohorts from Section 4: short ($T \leq 8$), medium ($8 < T \leq 20$), and long ($T > 20$), where T is each task's median full-history trajectory length. Assignment is held fixed across memory policies. A policy's own trajectory length is an outcome, not the variable used to select its cohort.

State-only success is lower in every cohort. The largest deficit occurs in the long cohort: 19.8% against 61.7% for full history. Retaining $k = 6$ restores 58.0% (Table 7). Its cost ratio is 0.66, compared with 1.30 for state alone. Cost per observed success is respectively \$0.1170 and \$0.4867, versus \$0.1899 for the control.

Long trajectories also strengthen the full-history cache advantage. Its median cache-hit rate rises from 71.1% in the short cohort to 91.2% in the long cohort; state alone and $k = 6$ reach 43.9% and 16.5% there. Cached-token volume alone cannot show this difference: hit rate also depends on uncached input.

These descriptive cohorts identify where a history window is useful, not a universal switching threshold. Short-cohort tasks can have long outlier repeats; workload-level efficiency and median matched costs therefore need not rank policies alike. Trajectories are policy-dependent, and no controlled intervention on task horizon establishes a causal crossover here.

Table 7: Control-defined trajectory-length cohorts: quality and cost. All retained outcomes are included. Cost ratio is the median same-task, same-repeat ratio to full history.

Approach	Solved rate (%)	Median T	Median cost ($)	Cost ratio (×)	Solved/$
Short: control median $T \leq 8$					
Full history	47.3	5	0.0086	1.00	6.89
Model-written state	29.0	4	0.0089	1.01	12.36
State + history ($k = 2$)	36.6	6	0.0088	1.02	10.70
State + history ($k = 6$)	46.2	6	0.0097	1.06	10.55
Medium: control median $8 < T \leq 20$					
Full history	75.9	11	0.0089	1.00	33.54
Model-written state	55.2	35	0.0283	3.48	9.88
State + history ($k = 2$)	64.4	14	0.0113	1.59	17.99
State + history ($k = 6$)	73.6	10	0.0100	1.48	28.37
Long: control median $T > 20$					
Full history	61.7	36	0.0523	1.00	5.27
Model-written state	19.8	101	0.0852	1.30	2.05
State + history ($k = 2$)	34.6	47	0.0585	0.91	4.25
State + history ($k = 6$)	58.0	21	0.0414	0.66	8.55

Table 8: The same trajectory-length cohorts: median context and token measurements. Cache hit is the median per-execution fraction $K/(U + K)$; token columns are separate medians.

Approach	Peak prompt (ktok)	Uncached in (ktok)	Cached in (ktok)	Output (ktok)	Cache hit (%)
Short: control median $T \leq 8$					
Full history	5.6	6.1	9.9	16.4	71.1
Model-written state	2.8	5.4	5.2	16.5	54.7
State + history ($k = 2$)	4.1	9.8	6.9	16.4	40.5
State + history ($k = 6$)	8.8	21.1	7.1	16.4	20.5
Medium: control median $8 < T \leq 20$					
Full history	10.7	20.9	44.4	6.7	79.6
Model-written state	4.0	31.3	58.0	39.4	52.2
State + history ($k = 2$)	6.1	28.7	18.2	16.7	39.7
State + history ($k = 6$)	9.4	38.6	12.9	8.3	25.1
Long: control median $T > 20$					
Full history	48.2	87.7	790.0	30.4	91.2
Model-written state	4.8	131.4	139.6	89.9	43.9
State + history ($k = 2$)	9.1	125.3	65.9	56.1	34.3
State + history ($k = 6$)	17.2	153.0	29.2	36.7	16.5

9 Can schema or protocol changes replace history retention?

A state-based agent might lose capability because its schema fits the task poorly or because a separate update tool changes the intended runtime contract. The hybrid comparison tests these alternatives by crossing the update interface with the semantic schema while leaving recent history absent (Table 9).

Table 9: Update protocol and schema within hybrid state, in the same two panels as Table 1. Hybrid aggregates cannot reconstruct matched cost ratios, peak prompt, or cache hit (–).

Approach	Solved rate (%)	Median T	Median cost ($)	Cost ratio (×)	Solved/$
Tool / Existing	39.7	8	0.0120	–	21.36
Inline / Existing	35.1	10	0.0165	–	15.25
Tool / Adapted	41.4	7	0.0137	–	21.11
Inline / Adapted	28.7	10	0.0190	–	9.92

Approach	Peak prompt (ktok)	Uncached in (ktok)	Cached in (ktok)	Output (ktok)	Cache hit (%)
Tool / Existing	–	14.5	9.3	17.3	–
Inline / Existing	–	10.6	1.9	28.6	–
Tool / Adapted	–	18.9	12.3	21.7	–
Inline / Adapted	–	14.1	2.4	33.1	–

The task-adapted schema changes success from 39.7% to 41.4%, a +1.7 percentage-point increase, while median cost rises from $0.0120 to $0.0137. This is a small observed gain, not a recovery comparable to retaining the larger history window. Its cost per success, $0.0474, is also slightly above the reference value of $0.0468.

Inline updates have a less favorable outcome. With the existing schema, success falls to 35.1% and median cost rises to $0.0165. Combining inline updates with the adapted schema gives 28.7% success and $0.1008 per observed success. The descriptive protocol–schema difference-in-differences is −8.0 percentage points: the inline effect is more negative under the adapted schema. The available summaries do not support a task-level interaction test.

These comparisons do not support either tested change as a substitute for recent-history retention. They also show why matching the shape of a published patch–action interface is insufficient evidence of matching its behavior in a different agent. Inline output changes the relationship between state maintenance and actions, and its output-token median is higher in both schema conditions. The measurements do not isolate patch content, invalid updates, or another trajectory change as the cause. Better schemas or implementations remain possible; the specific interventions tested here do not remove the deficit.

10 What the evidence changes about memory design

10.1 A useful abstraction with distinct failure modes

SKILL.state separates working context from the transcript. The coding experiments support that architectural idea while showing why a smaller state is not a complete memory policy. Three measured effects matter. Sparse runtime records leave useful execution evidence absent; retaining commands and changes improves on file references alone. Removing recent interactions

increases the pilot's repetition diagnostic and lowers success in the larger comparisons. Changing state and history also changes cache reuse, so logical token reduction is not a price comparison.

The remedies address different parts of this problem. Runtime ownership avoids asking the model to rewrite every observed fact. A recent window preserves detail that a schema did not select. Their combination still requires evaluation: enriching the runtime record did not produce monotone gains, and history recovery had different cost consequences in the two implementations. The evidence supports a division of memory responsibilities, not a claim that the tested hybrid dominates full history.

The extended window sweep turns $k = 6$ from an unexplained constant into a measured threshold: success and repeated invocation stop improving there, but at the ceiling and floor of a 12-task pilot. Wider windows keep adding prompt without removing the per-turn cost of rewriting state. Some lost information therefore concerns the local execution sequence; older evidence may still matter on tasks that leave headroom, and a late-relevance task could need an observation outside every finite window.

10.2 Why fixed-horizon savings do not settle this question

The original synthetic scaling experiments hold the event stream fixed; our agent chooses actions and when to stop [Badhe et al., 2026]. A context bound at a fixed horizon does not constrain the extra work a memory policy may induce. The original paper also reports public interactive benchmarks, so this distinction is not an explanation of all differences between the studies. Different models, schemas, and runtimes remain alternative explanations. Its reported total-token savings likewise cannot determine monetary savings without input, output, and cache accounting. Supplementary S6 illustrates this missing information without claiming to reconstruct the authors' bill.

The horizon cohorts provide no universal rule to switch from transcript to state. State alone loses success in every cohort, whereas limited history recovers much of the gap even in the long-reference cohort. An adaptive policy should therefore respond to information needs and complete-task cost, not assume that trajectory age makes recent evidence dispensable. A retrievable archive and a matched history-only control are the next tests: the former checks recovery of older evidence, the latter whether state adds value beyond a recent window.

Scope. These are implementation-specific findings on one model and task collection. Exploratory configuration selection, in-sample schema adaptation, unadjusted repeated-task tests, incomplete execution-source provenance, and aggregate-only hybrid reporting limit inference. The pilot supports descriptive resource and repetition comparisons, not a success-equivalence test. Neither causal forgetting nor the effectiveness of adaptive switching and retrieval is established.

11 Conclusion

Structured state is useful for bounding active context, but replacing history changes what the agent can do and how much work it performs. The experiments identify partial remedies: execution records beyond file references improve exploratory hybrid memory, and recent interactions recover much of the success lost by history replacement. Cache-aware accounting and control-defined horizon cohorts show why neither prompt size nor a single aggregate cost captures this trade-off.

The complete window sweep locates the threshold between two and six recent messages. Beyond six, success and repeated invocation hit the resolution limit of the pilot while prompt size keeps growing and the per-turn cost of rewriting state stays; the pilot cannot rank $k = 6$,

12, and ∞. The practical design principle is to separate runtime-recorded evidence, model-written interpretation, and access to original interactions. Compression is valuable when this combination preserves the ability to finish tasks; adaptive switching and retrieval must still demonstrate that benefit.

References

Sanket Badhe, Priyanka Tiwari, and Jonghyun Chung. SKILL.state: Scalable long-horizon agent skills. *arXiv preprint arXiv:2608.26263*, 2026. URL `https://arxiv.org/abs/2608.26263`. v3, 02 Sep 2026.

Mike A. Merrill et al. Terminal-Bench: Benchmarking agents on hard, realistic tasks in command line interfaces, 2026. URL `https://arxiv.org/abs/2601.11868`. arXiv:2601.11868. Version 2.0 used in this work.

Supplementary material

S1 Data, metric definitions, and what is missing

The artifact contains execution-level records for model-written state and for the runtime-record comparison, final aggregate summaries for the hybrid comparison, and finalized summaries for the window pilot. All tables and numerical claims are generated from these inputs. Hybrid summaries contain solved-rate margins and matched discordant counts, enough for the reported contrasts and exact McNemar tests, but not trajectories, per-call prompts, or cache hit. The pilot's raw sessions are unavailable; its task-level outcomes are transcribed from the research log.

Conditional resource ratios restrict to shared successes:

$$\text{median}_{(i,r):\, Y_{iar}=Y_{i0r}=1} \frac{X_{iar}}{X_{i0r}}.$$

A zero denominator leaves the ratio undefined. Efficiency uses all retained outcomes:

$$\text{Solved/\$} = \frac{\sum Y}{\sum D}, \qquad \text{Solved/Mtok} = \frac{10^6 \sum Y}{\sum (U + K + O)}.$$

Solved/Mtok counts logical tokens, not equal work across cached and uncached input. Hybrid efficiencies are approximate because their cost totals are rounded.

S2 Matched outcome tests

Table S1: Solved-rate differences in percentage points with exact McNemar tests. Bootstrap intervals are available only for model-written memory.

Approach	Δ (pp)	95% CI (pp)	p
Model-written memory vs. full history			
Model-written state	-26.4	[-34.1, -18.8]	2.1×10^{-13}
State + history ($k = 2$)	-16.1	[-23.4, -8.8]	9.7×10^{-7}
State + history ($k = 6$)	-2.3	[-7.7, +3.4]	0.488
Hybrid memory vs. full history			
Hybrid, no failure records	-13.2	–	4.3×10^{-4}
Hybrid state	-12.1	–	0.001
Hybrid + adapted schema	-10.3	–	0.008
Hybrid + inline	-16.7	–	5.7×10^{-5}
Hybrid + inline + adapted schema	-23.0	–	2.3×10^{-8}
Hybrid + history ($k = 2$)	-10.9	–	0.001
Hybrid + history ($k = 6$)	-3.4	–	0.392
Hybrid variants vs. hybrid state			
Hybrid, no failure records	-1.1	–	0.824
Hybrid + adapted schema	+1.7	–	0.678
Hybrid + inline	-4.6	–	0.185
Hybrid + inline + adapted schema	-10.9	–	0.001
Hybrid + history ($k = 2$)	+1.1	–	0.824
Hybrid + history ($k = 6$)	+8.6	–	0.011

S3 Cost over all retained executions

S4 Success-subset sensitivity of runtime-record costs

Variable-subset ratios use each configuration's own shared successes with full history; fixed-subset ratios use tasks solved by every configuration. The richest variants cost 0.92 and 0.94

Table S2: Solved-rate differences to full history within control-defined horizon cohorts (model-written memory).

Approach	Δ (pp)	p
Short: control median $T \leq 8$		
Model-written state	-18.3	2.2×10^{-4}
State + history ($k = 2$)	-10.8	0.031
State + history ($k = 6$)	-1.1	1.000
Medium: control median $8 < T \leq 20$		
Model-written state	-20.7	0.003
State + history ($k = 2$)	-11.5	0.076
State + history ($k = 6$)	-2.3	0.815
Long: control median $T > 20$		
Model-written state	-42.0	1.9×10^{-8}
State + history ($k = 2$)	-27.2	5.9×10^{-5}
State + history ($k = 6$)	-3.7	0.648

Table S3: Cost and efficiency including failures. Median and mean cost answer different questions from aggregate cost per success; none estimates retry-until-success cost.

Approach	Median cost ($)	Mean cost ($)	Cost/success ($)	Solved/$	Solved/Mtok
Model-written memory					
Full history	0.0137	0.0684	0.1116	8.96	0.421
Model-written state	0.0239	0.0568	0.1630	6.14	1.245
State + history ($k = 2$)	0.0151	0.0493	0.1091	9.16	1.964
State + history ($k = 6$)	0.0154	0.0453	0.0768	13.02	2.718
Hybrid memory (efficiencies approximate)					
Full history	0.0104	0.0245	0.0473	21.13	–
Hybrid state	0.0120	0.0186	0.0468	21.36	–
Hybrid + adapted schema	0.0137	0.0196	0.0474	21.11	–
Hybrid + inline	0.0165	0.0230	0.0656	15.25	–
Hybrid + inline + adapted schema	0.0190	0.0290	0.1008	9.92	–
Hybrid + history ($k = 2$)	0.0122	0.0189	0.0462	21.65	–
Hybrid + history ($k = 6$)	0.0144	0.0242	0.0501	19.95	–

times full history on the first, 1.03 and 1.17 on the second. The saving over file references survives; a saving over full history does not.

Table S4: Runtime-record cost sensitivity. Entries are medians of task-wise ratios to full history.

Runtime-owned information	Variable subset Cost	Fixed subset Cost	Fixed subset Turns	Fixed subset Output
File references	2.31×	2.50×	1.61×	4.03×
Files, changes, commands	1.20×	1.07×	0.77×	1.56×
+ diagnostics, environment, tests	0.92×	1.03×	0.75×	1.75×
+ failed-attempt records	0.94×	1.17×	0.79×	1.73×

S5 Window pilot per-repetition ratios

Table S5: Window pilot: median same-repetition ratios to full history for cost and peak prompt, reported separately for each repetition.

Approach	Cost ratio Repeat 1	Cost ratio Repeat 2	Cost ratio Repeat 3	Peak ratio Repeat 1	Peak ratio Repeat 2	Peak ratio Repeat 3
Model-written state	2.35	2.09	1.94	0.36	0.38	0.29
State + history ($k = 2$)	1.84	1.57	1.13	0.63	0.57	0.50
State + history ($k = 6$)	1.82	1.67	1.68	0.90	0.79	0.85
State + history ($k = 12$)	2.25	1.58	1.63	1.04	1.31	1.04
State + history ($k = \infty$)	1.96	2.00	1.55	1.40	1.69	1.31

S6 Cache-discount scenario for the original results

The original publication reports total tokens only. Multiplying its full/state token ratio by an assumed cache discount (full-history hit 0.93, state hit 0.72, cache price 0.198 of input) gives the proxy below. It treats all tokens as cacheable input and is a sensitivity illustration, not a reconstructed invoice.

Table S6: Published full/state token ratio and discounted-token proxy by event horizon. Values above one favor state under the proxy.

Event horizon (T)	Published token ratio (Full / State)	Discounted-token proxy (Full / State)
10	1.61×	0.97×
25	2.90×	1.74×
50	5.69×	3.42×
100	19.04×	11.45×
200	21.32×	12.82×